

AI Resume Builder - Understanding Client-Server Architecture

Objective

The objective of this task was to understand the communication between a frontend application and a backend API using HTML, CSS, JavaScript, and FastAPI.

Technologies Used

- HTML
 - CSS
 - JavaScript
 - FastAPI
 - Python
 - JSON
-

Project Overview

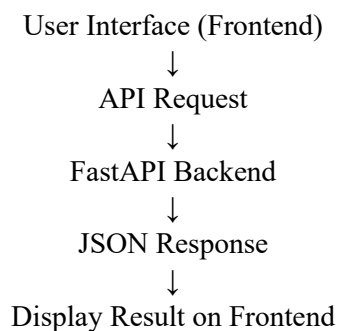
A simple AI Resume Builder application was created.

The application allows a user to enter:

- Name
- Job Role
- Skills

When the user clicks the "Generate Resume" button, the frontend sends the entered data to a FastAPI backend. The backend processes the data and returns a generated resume summary, which is then displayed on the webpage.

System Architecture



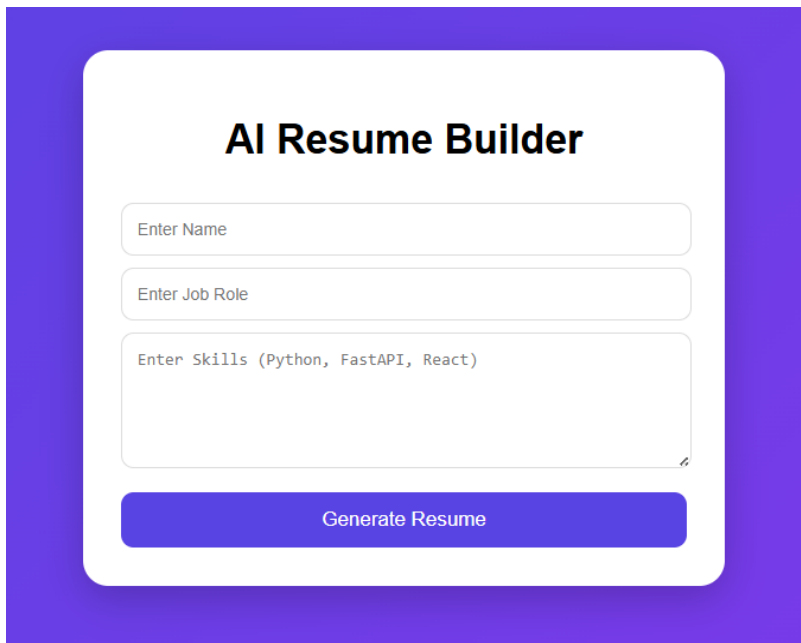
Frontend Implementation

The frontend was developed using HTML, CSS, and JavaScript.

Features:

- User-friendly interface
- Input fields for Name, Job Role, and Skills
- Generate Resume button
- Dynamic display of generated resume content

Screenshot 1: Frontend User Interface

A screenshot of a web application titled "AI Resume Builder". The interface is set against a solid purple background. In the center, there is a white rounded rectangle containing the form. The form has three input fields: "Enter Name", "Enter Job Role", and "Enter Skills (Python, FastAPI, React)". Below these fields is a prominent blue button labeled "Generate Resume".

AI Resume Builder

Enter Name

Enter Job Role

Enter Skills (Python, FastAPI, React)

Generate Resume

Backend Implementation

The backend was developed using FastAPI.

Features:

- POST endpoint: /generate
- Receives user data
- Generates a resume summary
- Returns response in JSON format

Screenshot 2: FastAPI Source Code

```
C:\Users\Admin\Desktop> API> main3.py generate
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from pydantic import BaseModel
4
5 app = FastAPI()
6
7 app.add_middleware(
8     CORSMiddleware,
9     (function) allow_credentials: bool
10     allow_credentials=True,
11     allow_methods=["*"],
12     allow_headers=["*"],
13 )
14
15 class ResumeData(BaseModel):
16     name: str
17     role: str
18     skills: str
19
20 @app.post("/generate")
21 def generate(data: ResumeData):
22     return [{"response": f"{data.name} is a {data.role} with skills in {data.skills}. Passionate about building innovative and scalable applications."}]
23
24
```

API Testing

The API was tested using FastAPI Swagger Documentation.

Endpoint Used:

POST /generate

The API successfully accepted input data and returned the generated response.

Screenshot 3: FastAPI Swagger Interface

FastAPI 0.1.0 OAS 3.1
[/openapi.json](#)

default ^

POST /generate Generate ^

Parameters Cancel Reset

No parameters

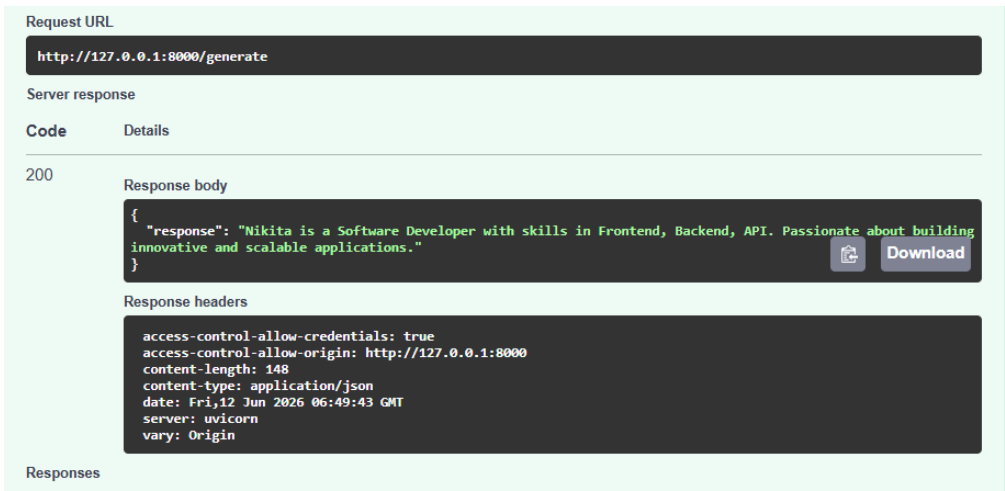
Request body required application/json v

Edit Value Schema

```
{
  "name": "Nikita",
  "role": "Software Developer",
  "skills": "Frontend, Backend, API"
}
```

Execute

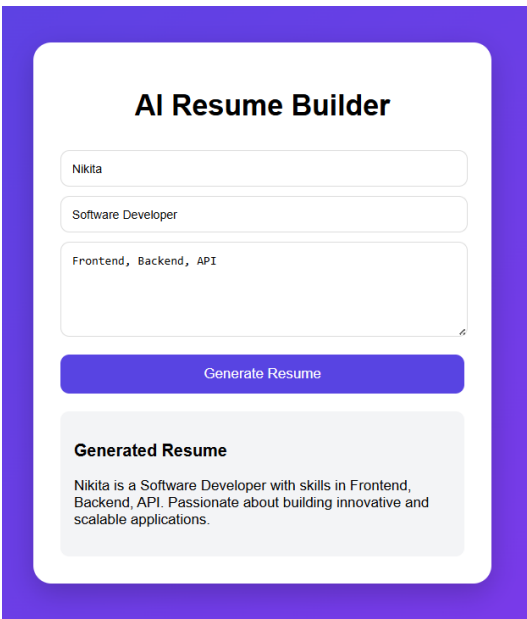
Screenshot 4: Successful API Response



Request and Response Flow

1. User enters Name, Role, and Skills.
2. User clicks "Generate Resume".
3. JavaScript sends a POST request to the FastAPI backend.
4. Backend receives the data.
5. Backend generates a resume summary.
6. JSON response is returned.
7. Frontend displays the generated resume.

Screenshot 5: Generated Resume Output



Sample Input

Name: Nikita

Role: Software Developer

Skills: Frontend, Backend, API

Sample Output

"Nikita is a Software Developer with skills in Frontend, Backend, API. Passionate about building innovative and scalable applications."

Learning Outcomes

Through this task, the following concepts were understood:

- Client-Server Architecture
 - REST API Communication
 - FastAPI Endpoint Creation
 - Handling JSON Data
 - JavaScript Fetch API
 - Frontend and Backend Integration
 - CORS Configuration
 - Request-Response Lifecycle
-

Conclusion

The AI Resume Builder successfully demonstrated communication between a frontend application and a FastAPI backend. The project helped in understanding how web applications exchange data through APIs and provided a foundation for building more advanced AI-powered applications in the future.